Spring Boot — Lecture 9

Spring Boot IoC — Auto-Configuration, @SpringBootApplication & Starters

1. Spring Core vs Spring Boot — What Changes

Everything you learned in Spring Core still applies in Spring Boot — same IoC Container, same DI, same bean lifecycle. The difference is how much manual configuration you need to write.

Task	Spring Core (manual)	Spring Boot (automatic)
Start IoC Container	new AnnotationConfigApplicationContext(AppConfig.class)	SpringApplication.run() does it automatically
Configuration file	Must create AppConfig.java with @Configuration + @ComponentScan	Not needed — main class itself is the config
Component scanning	@ComponentScan("in.strikcs") in AppConfig	Auto-scans the package where the main class lives
Web server (Tomcat)	Must configure manually — lots of XML/annotations	Just add spring-boot-starter-web dependency — Tomcat starts automatically
Dependency version management	Must find compatible versions manually for every library	spring-boot-starter-parent manages all versions for you
Get a bean	context.getBean(OrderService.class)	Use @Autowired injection — no need to touch the context

2. Creating a Spring Boot Project — Two Ways

Method	Steps	When to use
Spring Initializr (start.spring.io)	Select Java, Maven, Spring Boot version, Group/Artifact IDs, Java version → Generate → Unzip → Open in IDE	Recommended — gives you a complete pre-configured project
Manual Maven project	Create Maven project in IntelliJ → add spring-boot-starter dependency + parent tag in pom.xml manually	When you want full control of the pom.xml

```
org.springframework.boot
spring-boot-starter-parent
4.1.0




    org.springframework.boot
    spring-boot-starter
```

The parent tag eliminates version conflicts. When you add any spring-boot- dependency, the parent already knows which version of that library is compatible with the current Spring Boot version. No more dependency version hunting.*

3. @SpringBootApplication — The Magic Annotation

@SpringBootApplication is a composed annotation — it contains three annotations inside it. Writing @SpringBootApplication is equivalent to writing all three.

```
// What @SpringBootApplication contains internally:
@SpringBootConfiguration       // = @Configuration (this class is a config file)
@EnableAutoConfiguration       // enables Spring Boot's auto-configuration magic
@ComponentScan                 // scans the current package for @Component classes
public @interface SpringBootApplication { ... }

// So this:
@SpringBootApplication
public class MyApplication { ... }

// Is EXACTLY the same as:
@SpringBootConfiguration
@EnableAutoConfiguration
@ComponentScan
public class MyApplication { ... }
```

Three annotations explained:

Annotation	What it does	Spring Core equivalent
@SpringBootConfiguration	Marks this class as a configuration class. You can write @Bean methods here. The main class itself IS the AppConfig — no separate AppConfig.java needed.	@Configuration on AppConfig.java
@ComponentScan	Scans for @Component classes. By convention, scans the package where the main class lives. All @Component classes in that package (and sub-packages) get managed by IoC Container.	@ComponentScan("in.strikcs") in AppConfig
@EnableAutoConfiguration	Enables Spring Boot's auto-configuration. Spring Boot automatically creates beans for standard libraries (Tomcat, Jackson, JPA etc.) based on what dependencies you added.	No equivalent in Spring Core — this is Spring Boot-only

4. Main Class = Configuration File

Because @SpringBootApplication includes @SpringBootConfiguration (which is like @Configuration), you can write @Bean methods directly inside the main class.

```
@SpringBootApplication
public class SpringBootCoreDemoApplication {

    public static void main(String[] args) {
        ApplicationContext context =
            SpringApplication.run(SpringBootCoreDemoApplication.class, args);

        OrderService order = context.getBean(OrderService.class);
        order.placeOrder();
    }

    // You can write @Bean methods directly here — no separate AppConfig.java needed
    @Bean
    public UserService getUserServiceBean() {
        return new UserService();
    }
}

// ■■■ OrderService.java ■■■
@Component
public class OrderService {
```

```
    private PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }

    public void placeOrder() {
        paymentService.pay();
        System.out.println("Order placed");
    }
}

// ■■■ PaymentService.java ■■■
@Component
public class PaymentService {
    public void pay() {
        System.out.println("Payment done");
    }
}
```

Do NOT use context.getBean() in Spring Boot applications — that is the Spring Core style. In Spring Boot, use @Autowired injection or constructor injection. Let Spring Boot handle everything. The getBean() approach is shown here only for comparison.

5. @ComponentScan — Package Convention

Spring Boot scans the package where the main class lives (and all sub-packages). This is the most important convention to follow.

Scenario	Will Spring Boot manage it?
OrderService.java in same package as main class	YES — scanned automatically
PaymentService.java in a sub-package of main package	YES — sub-packages are included
SomeService.java in a sibling package outside main package	NO — not scanned by default
SomeService.java in parent package of main package	NO — only downward scanning, not upward

```
// Project structure — what gets scanned:
src/main/java/
  in/strikcs/springbootcoredemo/          ← main class lives here
    SpringBootCoreDemoApplication.java    ← @SpringBootApplication
    OrderService.java                     ← @Component ✓ scanned
    PaymentService.java                   ← @Component ✓ scanned
    payment/
      UpiPaymentService.java              ← @Component ✓ scanned (sub-package)
  in/strikcs/other/                       ← sibling package
    SomeService.java                      ← @Component ✗ NOT scanned

// To override the default scan location:
@SpringBootApplication(scanBasePackages = "in.strikcs")
public class SpringBootCoreDemoApplication { ... }
// Now ALL of in.strikcs is scanned, including sibling packages
```

6. SpringApplication.run() — IoC Container Startup

SpringApplication.run() is the entry point. It starts the IoC Container and returns a ConfigurableApplicationContext (which extends ApplicationContext).

```
@SpringBootApplication
public class MyApp {
    public static void main(String[] args) {
```

```
        // SpringApplication.run() starts the IoC Container and returns it
        ApplicationContext context =
            SpringApplication.run(MyApp.class, args);

        // From this point, all beans are ready to use
        // (This is Spring Core style — in real Spring Boot apps, use @Autowired instead)
    }
}

// What SpringApplication.run() does internally:
// 1. Creates the IoC Container (ApplicationContext)
// 2. Reads configuration from @SpringBootApplication on the passed class
// 3. Performs component scanning (finds @Component classes)
// 4. Applies auto-configuration (creates beans for detected dependencies)
// 5. Returns the ready ApplicationContext
```

7. @EnableAutoConfiguration — How Auto-Configuration Works

This is the most powerful Spring Boot feature. Spring Boot automatically creates beans for standard libraries based on what you added to pom.xml — without you writing any configuration.

Three ways beans get created in Spring Boot:

Method	Who writes it	Example
@Component on a class	You (the developer)	Your own OrderService, PaymentService classes
@Bean in a @Configuration class	You (the developer)	Third-party class you want Spring to manage: @Bean public JacksonParser getParser()
@AutoConfiguration (internal)	Spring Boot itself (pre-written)	Tomcat, Jackson, JPA, DataSource etc. — created automatically when you add dependencies

How @AutoConfiguration works internally:

```
// Example of what Spring Boot has PRE-WRITTEN inside its own code:

@AutoConfiguration                        // Spring Boot's version of @Configuration
@ConditionalOnClass(WebMvcConfigurer.class)  // Only activate IF this class is on classpath
public class WebMvcAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean             // Only create if user hasn't created their own
    public WebMvcConfigurer defaultWebMvcConfig() {
        return new WebMvcConfigurer() { ... };
    }

    @Bean
    public DispatcherServlet dispatcherServlet() {
        return new DispatcherServlet();
    }
    // ... many more beans
}

// YOU never see this code — it's inside the spring-boot-autoconfigure JAR.
// Spring Boot runs it automatically when @EnableAutoConfiguration is active.
```

Key annotations used in auto-configuration:

Annotation	Meaning
@AutoConfiguration	Marks a class as an auto-configuration source. Spring Boot discovers and runs these automatically at startup when @EnableAutoConfiguration is active.

Annotation	Meaning
@ConditionalOnClass(SomeClass.class)	Only activate this auto-configuration IF SomeClass is on the classpath. Example: TomcatAutoConfiguration only runs if Tomcat JAR is present.
@ConditionalOnMissingBean	Only create this bean IF the user hasn't already created one themselves. Prevents overriding user's custom beans with Spring Boot defaults.

8. Auto-Configuration in Action — Tomcat Example

The most visible example of auto-configuration: adding spring-boot-starter-web automatically starts an embedded Tomcat server.

```
// BEFORE adding spring-boot-starter-web:
// pom.xml has only: spring-boot-starter
// Result when you run: NO Tomcat, just prints output and exits
// Console: "Started SpringBootCoreDemoApplication in 0.3 seconds"

// AFTER adding spring-boot-starter-web:

    org.springframework.boot
    spring-boot-starter-web



// Result when you run: Tomcat starts automatically on port 8080!
// Console: "Tomcat started on port 8080 (http) with context path '/'"
// Console: "Started SpringBootCoreDemoApplication in 1.8 seconds"

// WHY? spring-boot-starter-web includes Tomcat JAR on the classpath.
// Spring Boot's TomcatAutoConfiguration sees Tomcat on classpath
// (@ConditionalOnClass) → creates all Tomcat beans → server starts.
// YOU wrote zero configuration for this.
```

This is the power of auto-configuration: add a dependency → Spring Boot detects it → creates all necessary beans → feature works automatically.

9. spring-boot-starter-parent — Dependency Version Management

The parent POM manages compatible versions for all Spring Boot ecosystem dependencies. You only need to specify groupId and artifactId — version comes from the parent.

```


       org.springframework.boot
       spring-boot-starter-web



       org.springframework.boot
       spring-boot-starter-data-jpa



       org.springframework.boot
       spring-boot-starter-test

       test
```

The parent POM is a hierarchy: your pom → spring-boot-starter-parent → spring-boot-dependencies (contains all version numbers). This is how Spring Boot achieves 'convention over configuration' for dependencies.

10. Spring Boot Project Structure

```
src/
  main/
    java/
      in/strikcs/springbootcoredemo/
        SpringBootCoreDemoApplication.java  ← @SpringBootApplication + main()
        OrderService.java                   ← @Component
        PaymentService.java                 ← @Component
    resources/
      application.properties               ← configuration (port, DB url, etc.)
      static/                              ← static web files (HTML, CSS, JS)
      templates/                           ← Thymeleaf/FreeMarker templates
  test/
    java/
      SpringBootCoreDemoApplicationTests.java  ← JUnit tests

pom.xml                                    ← Maven build file with parent + dependencies
```

11. Full Comparison — Spring Core vs Spring Boot

What you write	Spring Core	Spring Boot
pom.xml	spring-context dependency with explicit version	spring-boot-starter + spring-boot-starter-parent (no versions needed)
IoC Container startup	new AnnotationConfigApplicationContext(AppConfig.class) — 1 line you write	SpringApplication.run(MyApp.class, args) — 1 line, already in the template
Configuration class	Must create AppConfig.java with @Configuration + @ComponentScan	No separate file — @SpringBootApplication on main class covers it
Component scanning	Must specify package: @ComponentScan("in.strikcs")	Auto-scans current package — just keep files in the right place
Third-party library beans	Must create @Bean methods in AppConfig for every library class you need	Auto-configuration creates them — just add the dependency
Web server	Must configure Tomcat/Jetty manually — complex setup	Add spring-boot-starter-web — Tomcat starts on port 8080 automatically
All business code	@Component, @Autowired, @Bean — SAME	@Component, @Autowired, @Bean — EXACTLY THE SAME

Key insight: Spring Boot does NOT change how beans work or how DI works. Everything from Spring Core applies. Spring Boot only eliminates the boilerplate setup. That is why understanding Spring Core first makes Spring Boot trivial.

Next Lecture: application.properties, CommandLineRunner, and proper Spring Boot bean injection patterns